Introduction to Machine Learning: Study Notes

Original study notes written as sample content for the Document Intelligence Assistant. Free to reuse.

1. What Is Machine Learning?

Machine learning (ML) is the study of algorithms that improve at a task through experience, usually by learning patterns from data rather than following hand-written rules. Tom Mitchell's widely quoted definition (1997) says a program learns from experience E with respect to a task T and performance measure P if its performance at T, as measured by P, improves with experience E.

Three broad families of machine learning are:

- **Supervised learning:** the model learns from labelled examples, pairs of inputs and correct outputs. Examples: predicting house prices (regression) or classifying emails as spam or not spam (classification).
- **Unsupervised learning:** the model finds structure in unlabelled data. Examples: grouping customers into segments (clustering) or reducing the number of features (dimensionality reduction).
- **Reinforcement learning:** an agent learns by taking actions in an environment and receiving rewards or penalties. Examples: game-playing agents and robot control.

Self-supervised learning, in which the labels are created from the data itself (for example, predicting the next word in a sentence), is the main way large language models are pre-trained.

2. The Basic Workflow

1. Define the problem and how success will be measured.
2. Collect and clean data; handle missing values and outliers.
3. Split the data into training, validation and test sets (a common split is 70/15/15 or 80/10/10).
4. Engineer or select features.
5. Train candidate models on the training set.
6. Tune hyperparameters using the validation set or cross-validation.
7. Evaluate the final model once on the test set.
8. Deploy, then monitor for data drift and degraded performance.

The test set must not be used for any decisions during development. Using it repeatedly leaks information and makes the final evaluation too optimistic.

3. Common Algorithms

Algorithm	Type	Key idea	Strengths	Weaknesses
Linear regression	Supervised, regression	Fits a straight line (or hyperplane) minimizing squared error	Fast, interpretable	Misses non-linear relationships
Logistic regression	Supervised, classification	Models the probability of a class with the sigmoid function	Interpretable, good baseline	Linear decision boundary
k-nearest neighbours	Supervised	Predicts from the k most similar training examples	Simple, no training phase	Slow at prediction time on large datasets; sensitive to feature scale
Decision tree	Supervised	Splits data with a sequence of if-then rules	Easy to explain, handles mixed data	Overfits easily
Random forest	Supervised, ensemble	Averages many trees trained on random samples and features	Strong default, robust	Less interpretable, larger models
Gradient boosting (e.g. XGBoost, LightGBM)	Supervised, ensemble	Adds trees one at a time, each correcting the previous errors	Often best on tabular data	Needs careful tuning
k-means	Unsupervised, clustering	Assigns points to the nearest of k centroids, then updates centroids	Simple, scalable	Must choose k; assumes round clusters
PCA	Unsupervised, dimensionality reduction	Projects data onto directions of maximum variance	Reduces noise and dimensions	Components can be hard to interpret
Neural networks	Supervised or self-supervised	Layers of weighted sums and non-linear activations trained by backpropagation	Excellent on images, text, audio	Need much data and compute

4. Overfitting and Underfitting

A model **overfits** when it learns noise in the training data and performs well on training data but poorly on new data. A model **underfits** when it is too simple to capture the underlying pattern and performs poorly on both.

This is often described as the **bias-variance trade-off**. High bias (overly simple assumptions) leads to underfitting; high variance (excessive sensitivity to the particular training set) leads to overfitting.

Ways to reduce overfitting:

- Collect more training data.

- Use a simpler model or fewer features.
- Regularization: L1 (lasso) adds the sum of absolute weights to the loss and can drive some weights to exactly zero; L2 (ridge) adds the sum of squared weights and shrinks weights smoothly.
- Dropout in neural networks, which randomly disables a fraction of units during training.
- Early stopping: stop training when validation error starts to rise.
- Cross-validation to get a more reliable estimate of performance.

5. Evaluation Metrics

5.1 Classification

A confusion matrix for a binary classifier counts true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN).

Metric	Formula	Answers the question
Accuracy	(TP + TN) / all predictions	What share of predictions were correct?
Precision	TP / (TP + FP)	Of the items predicted positive, how many really were?
Recall (sensitivity)	TP / (TP + FN)	Of the real positives, how many did we find?
F1 score	2 x precision x recall / (precision + recall)	Harmonic mean of precision and recall
ROC AUC	Area under the ROC curve	How well does the model rank positives above negatives?

Accuracy can be misleading on imbalanced data. If 99% of transactions are legitimate, a model that always predicts "legitimate" is 99% accurate but useless for detecting fraud. Precision and recall are more informative in such cases.

Worked example: a spam filter flags 50 emails. 40 are really spam (TP = 40, FP = 10). There were 60 spam emails in total, so 20 were missed (FN = 20). Precision = 40 / 50 = 0.80. Recall = 40 / 60 = 0.67. F1 = 2 x 0.80 x 0.67 / (0.80 + 0.67), about 0.73.

5.2 Regression

- **MAE** (mean absolute error): average absolute difference between predictions and actual values; same units as the target.
- **RMSE** (root mean squared error): penalizes large errors more heavily than MAE.
- **R squared**: share of the variance in the target explained by the model; 1.0 is a perfect fit.

6. Cross-Validation

In **k-fold cross-validation** the training data is split into k equal folds (commonly k = 5 or 10). The model is trained k times, each time holding out a different fold for validation, and the k scores are averaged. Stratified k-fold keeps the class proportions the same in each fold, which matters for imbalanced classification. For time series, use forward-chaining splits so the model is never validated on data from before its training period.

7. Neural Networks and Deep Learning

A neural network is made of layers of units. Each unit computes a weighted sum of its inputs plus a bias, then applies a non-linear **activation function** such as ReLU (max(0, x)), sigmoid or tanh. Without non-linear activations, any stack of layers would collapse into a single linear model.

Training uses **gradient descent**: the loss is computed on a batch of examples, **backpropagation** computes the gradient of the loss with respect to every weight using the chain rule, and the weights are moved a small step in the opposite direction of the gradient. The step size is the **learning rate**. Too high a learning rate makes training diverge; too low makes it very slow. Optimizers such as Adam adapt the step size for each weight.

Important architectures:

- **Convolutional neural networks (CNNs):** use small learned filters slid across an image; well suited to vision tasks.
- **Recurrent neural networks (RNNs) and LSTMs:** process sequences one step at a time while keeping a hidden state.
- **Transformers:** introduced in the 2017 paper "Attention Is All You Need" by Vaswani and colleagues. They use self-attention to relate every token in a sequence to every other token in parallel, and are the basis of modern large language models.

8. Embeddings and Retrieval

An **embedding** is a vector of numbers that represents an item (a word, sentence, image or document) so that similar items have nearby vectors. Similarity is often measured with **cosine similarity**, the cosine of the angle between two vectors, which ranges from -1 to 1.

Retrieval-augmented generation (RAG) uses embeddings to find passages relevant to a question and passes them to a language model, so answers can be grounded in specific documents rather than only in what the model memorized during training.

9. Glossary

- **Feature:** an input variable used by a model.
- **Label:** the correct output for a training example.
- **Hyperparameter:** a setting chosen before training, such as the learning rate or the number of trees.
- **Epoch:** one full pass through the training data.
- **Batch size:** number of examples used to compute one gradient update.
- **Data drift:** a change in the distribution of input data after deployment.
- **Data leakage:** when information that would not be available at prediction time is used during training.